

Aula 07 — Pré-processamento e *Pipelines*

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: ISLP labs cap. 5–6; AME §2.1.2

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- explicar por que o pré-processamento faz parte do *modelo*, e não dos dados;
- definir a **padronização** (`StandardScaler`) e listar suas vantagens e desvantagens;
- identificar e evitar o **vazamento de dados** (*data leakage*);
- construir um ***pipeline*** que encadeia pré-processamento e estimador como um único objeto;
- combinar *pipeline* com validação cruzada e busca de hiperparâmetros sem vazamento;
- conhecer outros pré-processamentos comuns (imputação, codificação de categóricas) e o `ColumnTransformer`.

Em sala

Esta é a aula que costura as anteriores em *boas práticas de implementação*. Vários métodos já exigiram padronização: Lasso/Ridge (Aula 02) e KNN (Aula 04); e a Aula 03 já avisou sobre vazamento. Aqui formalizamos *como* fazer isso certo. Pergunta de abertura: “*onde, exatamente, eu devo calcular a média e o desvio para padronizar: antes ou depois de separar treino e teste?*”

1 Motivação: o pré-processamento é parte do modelo

Na prática, raramente alimentamos o estimador com os dados brutos. Um fluxo típico encadeia várias transformações: padronizar, imputar faltantes, codificar variáveis categóricas, reduzir dimensão (PCA), e só então ajustar o modelo. A tentação é aplicar essas transformações “de uma vez” no conjunto inteiro — e é exatamente aí que mora o erro.

Ideia central

O insight central da aula: **toda transformação que “aprende” algo dos dados** (uma média, um desvio, as categorias presentes, os componentes principais) é *parte do procedimento de estimação*. Logo, deve ser *ajustada apenas no treino e aplicada ao teste/validação* — nunca o contrário.

A ferramenta do `scikit-learn` para isso é o ***Pipeline***: ele agrega vários processamentos e o estimador final em um único bloco que se comporta como um estimador (`fit/predict`), garantindo que cada etapa veja apenas os dados corretos.

2 O StandardScaler (padronização)

Cada coluna $\mathbf{X}_i$ de uma base é uma amostra de uma variável aleatória X_i com $\mu_i = \mathbb{E}[X_i]$ e $\sigma_i^2 = \text{Var}(X_i)$ (supostos finitos). A **padronização** transforma

$$\tilde{X}_i = \frac{X_i - \mu_i}{\sigma_i}, \quad (1)$$

de modo que cada covariável passa a ter média 0 e variância 1. Na prática, μ_i e σ_i são desconhecidos e *estimados* pela média e desvio amostrais — e é justamente esse “estimados” que cria o risco de vazamento.

2.1 Vantagens

- Covariáveis tornam-se **adimensionais**: somem as unidades (kg, R\$, anos).
- Resolve **discrepâncias numéricas** entre escalas — crucial para métodos baseados em distância (KNN, Aula 04) e em penalização (Lasso/Ridge, Aula 02), que de outro modo seriam dominados pelas variáveis de maior magnitude.
- Torna o **intercepto desnecessário** em regressão linear (dados centrados em zero).

2.2 Desvantagens

- Leve perda de **interpretabilidade**: os coeficientes passam a estar em “desvios-padrão”, não nas unidades originais.
- μ_i e σ_i são **aprendidos dos dados** — exigindo cuidado para não vazar informação do teste (próxima seção).

3 Vazamento de dados (*data leakage*)

Atenção: A armadilha

Se você aplicar o **StandardScaler** (ou qualquer transformação que aprende parâmetros) no *conjunto inteiro* antes de separar treino e teste, a média/desvio usados no treino **já contém informação do teste**. As métricas deixam de medir o desempenho em dados *verdadeiramente novos* e ficam otimistas. Isso é **vazamento de dados**.

A regra correta:

1. separe treino e teste *primeiro*;
2. fit do scaler **só no treino** (aprende $\hat{\mu}$, $\hat{\sigma}$ do treino);
3. **transform** aplicado ao treino e ao teste com *esses mesmos* parâmetros.

Atenção: Vazamento dentro da validação cruzada

O mesmo vale para *cada dobra* da CV (Aula 03): o scaler deve ser reajustado usando apenas o treino *daquela dobra*. Padronizar antes da CV vaza o “treino futuro” de cada dobra. Fazer isso manualmente é trabalhoso e propenso a erro — por isso usamos *pipelines*, que cuidam disso automaticamente.

4 Pipelines

Um Pipeline encadeia etapas (nome, transformador) terminadas por um estimador. Ao chamar:

- `pipe.fit(X_tr, y_tr)`: cada transformador faz `fit_transform` no treino, em sequência, e o estimador final é ajustado;

- `pipe.predict(X_te)`: cada transformador apenas **transform** (com os parâmetros aprendidos no treino) e o estimador **prediz**.

Assim, é *impossível* vaziar o teste por construção: o teste nunca participa de nenhum **fit**.

```

1 from sklearn.preprocessing import StandardScaler
2 from sklearn.linear_model import Lasso
3 from sklearn.pipeline import Pipeline
4 from sklearn.model_selection import train_test_split, GridSearchCV
5
6 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)
7
8 pipe = Pipeline(steps=[("scaler", StandardScaler()),
9                        ("lasso", Lasso())])
10 pipe.fit(X_tr, y_tr) # scaler aprende mu/sigma SD do treino
11 pipe.predict(X_te) # teste e apenas transformado

```

4.1 Pipeline + validação cruzada + busca de hiperparâmetros

O grande payoff: passar o *pipeline* inteiro ao `GridSearchCV`. Em cada dobra, todo o pré-processamento é reajustado corretamente, e a busca pelos hiperparâmetros (inclusive do scaler, se houver) fica livre de vazamento. Note a sintaxe `nomeEtapa_parametro` para acessar parâmetros de cada etapa.

```

1 from sklearn.linear_model import ElasticNet
2
3 pipe_en = Pipeline([("scaler", StandardScaler()),
4                    ("enet", ElasticNet())])
5
6 param_grid = {"enet__alpha": [0.001, 0.01, 0.1, 1, 5, 10, 50, 100],
7              "enet__l1_ratio": [0.1, 0.5, 0.7, 0.9, 0.95, 0.99, 1]}
8
9 busca = GridSearchCV(pipe_en, param_grid, cv=5,
10                    scoring="neg_mean_squared_error")
11 busca.fit(X_tr, y_tr) # CV SEM vazamento, etapa por etapa
12 print("melhores parametros:", busca.best_params_)
13 print("risco no teste:", -busca.score(X_te, y_te))

```

Ideia central

Encare o *pipeline* como “o modelo”. Tudo o que você faria errado à mão — padronizar cedo demais, esquecer de aplicar a mesma transformação no teste, vaziar na CV — o *pipeline* previne por construção. É a tradução em código das regras das Aulas 02–04.

5 Outros pré-processamentos comuns

Imputação preencher valores faltantes (`SimpleImputer` com média, mediana ou moda). A estatística de imputação também se aprende *só no treino*.

Codificação de categóricas

`OneHotEncoder` (cria *dummies*) ou `OrdinalEncoder`. Lembre (Aula 06) que árvores lidam com categóricas sem isso.

Escalas alternativas

`MinMaxScaler` (intervalo $[0, 1]$), `RobustScaler` (baseado em quantis, resistente a *outliers*).

Redução de dimensão

PCA como etapa do *pipeline* (Aula extra), também ajustada apenas no treino.

Quando diferentes colunas precisam de tratamentos diferentes (numéricas padronizadas, categóricas codificadas), usa-se o `ColumnTransformer`:

```
1 from sklearn.compose import ColumnTransformer
2 from sklearn.preprocessing import OneHotEncoder, StandardScaler
3 from sklearn.impute import SimpleImputer
4
5 prep = ColumnTransformer([
6     ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
7                       ("sc", StandardScaler())]), colunas_numericas),
8     ("cat", OneHotEncoder(handle_unknown="ignore"), colunas_categoricas),
9 ])
10 modelo = Pipeline([("prep", prep), ("clf", Lasso())])
```

Resumo

- Pré-processamento que aprende parâmetros é **parte do modelo**: ajuste só no treino, aplique ao teste.
- **Padronização (1)**: adimensionaliza, equaliza escalas (essencial para KNN e penalização), dispensa intercepto; custo: leve perda de interpretabilidade.
- **Vazamento**: padronizar/imputar antes de separar treino/teste (ou antes de cada dobra) infla as métricas. Evite!
- **Pipeline**: encadeia tudo num único objeto e previne vazamento por construção; combine com `GridSearchCV`.
- `ColumnTransformer` aplica tratamentos distintos a colunas distintas.

Leitura recomendada. [ISLP] laboratórios dos Capítulos 5 e 6 (uso de *pipelines* e validação cruzada em Python); [AME] §2.1.2 (regressão linear na prática). Documentação do `scikit-learn`: *Pipelines and composite estimators*.